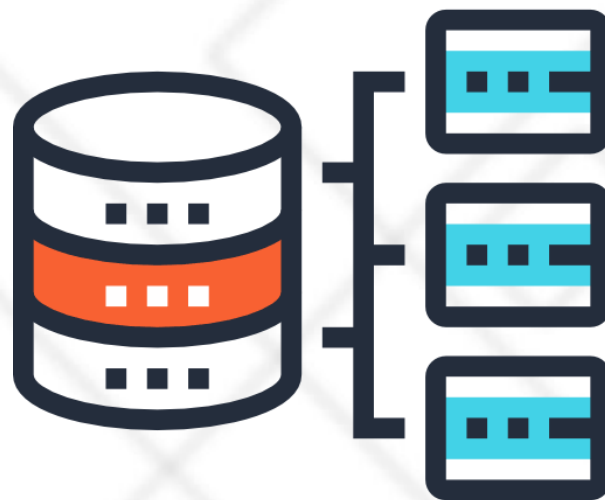




北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



第11章 数据库设计

北京理工大学 计算机学院

段春晖

duanch@bit.edu.cn

第11章 数据库设计



- 数据库设计方法概述
- 数据库设计过程（六个步骤）

1. 数据库设计方法

□什么是数据库设计

- 广义：数据库应用系统
- 狭义：数据库本身
- 数据库设计是指对于一个给定的应用环境
 - ◆ 构造优化的数据库逻辑模式和物理结构，并据此建立数据库及其应用系统
 - ◆ 使之能够有效地存储和管理数据，满足各种用户的应用需求，包括信息管理要求和数据操作要求
- 在数据库领域内，常常把使用数据库的各类系统统称为数据库应用系统
 - ◆ 管理信息系统，办公自动化系统，地理信息系统，电子政务系统，电子商务系统

1. 数据库设计方法

- 大型数据库的设计和开发是一项庞大的工程，涉及多学科的综合技术
- 数据库设计的特点
 - 涉及面广，包含计算机专业知识及业务系统专业知识；要解决技术及非技术两方面的问题
 - 涉及内容包括
 - ◆ 静态结构设计：数据库的模式框架设计
 - 概念数据模型、逻辑数据模型、物理数据模型（存储结构）
 - ◆ 动态行为设计（应用程序设计）
 - 功能组织、流程控制

1. 数据库设计方法

□ 数据库设计人员应该具备的技术和知识

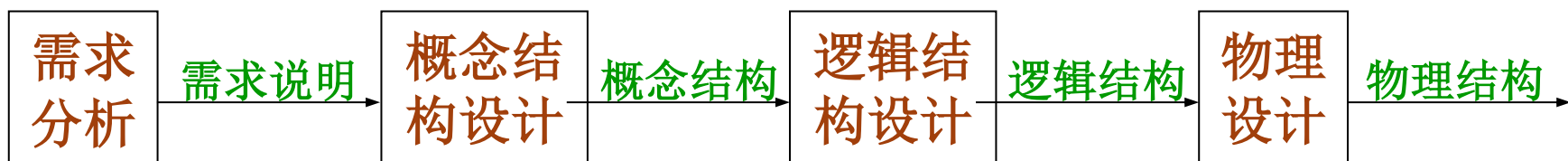
- 计算机的基本知识
- 软件工程的原理和方法
- 程序设计的方法和技巧
- 数据库的基本知识
- 数据库设计技术
- 应用领域的知识

1. 数据库设计方法

□ 规范设计法：本质上仍然是手工设计方法，基本思想是过程迭代和逐步求精

□ 数据库设计方法

- 新奥尔良 (New Orleans) 方法：数据库设计分四个阶段



- 基于E-R模型的数据库设计方法
- 基于3NF的设计方法：用关系数据理论为指导来设计数据库的逻辑模式
- ODL (Object Definition Language) 方法：面向对象的数据库设计方法

数据库设计的过程

□(1) 需求分析

- 准确了解与分析用户需求（包括数据与处理）
- 是整个设计过程的基础，是最困难、最耗时的一步

□(2) 概念结构设计

- 是整个数据库设计的关键
- 通过对用户需求进行综合、归纳与抽象，形成一个独立于具体DBMS的概念模型

□(3) 逻辑结构设计

- 将概念结构转换为某个DBMS所支持的数据模型
- 对其进行优化

□(4) 数据库物理设计阶段

- 为逻辑数据模型选取一个最适合应用环境的物理结构 (包括存储结构和存取方法)

□(5) 数据库实施阶段

- 运用DBMS提供的数据库语言、工具及宿主语言，根据逻辑设计和物理设计的结果
- 建立数据库、编制与调试应用程序、组织数据入库、并进行试运行

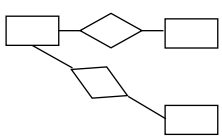
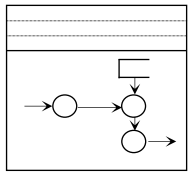
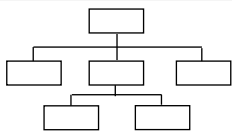
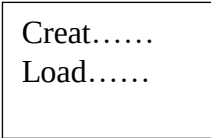
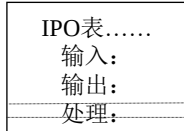
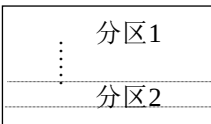
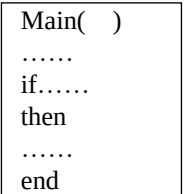
□(6) 数据库运行和维护

- 数据库应用系统经过试运行后即可投入正式运行
- 在数据库系统运行过程中必须不断地对其进行评价、调整与修改

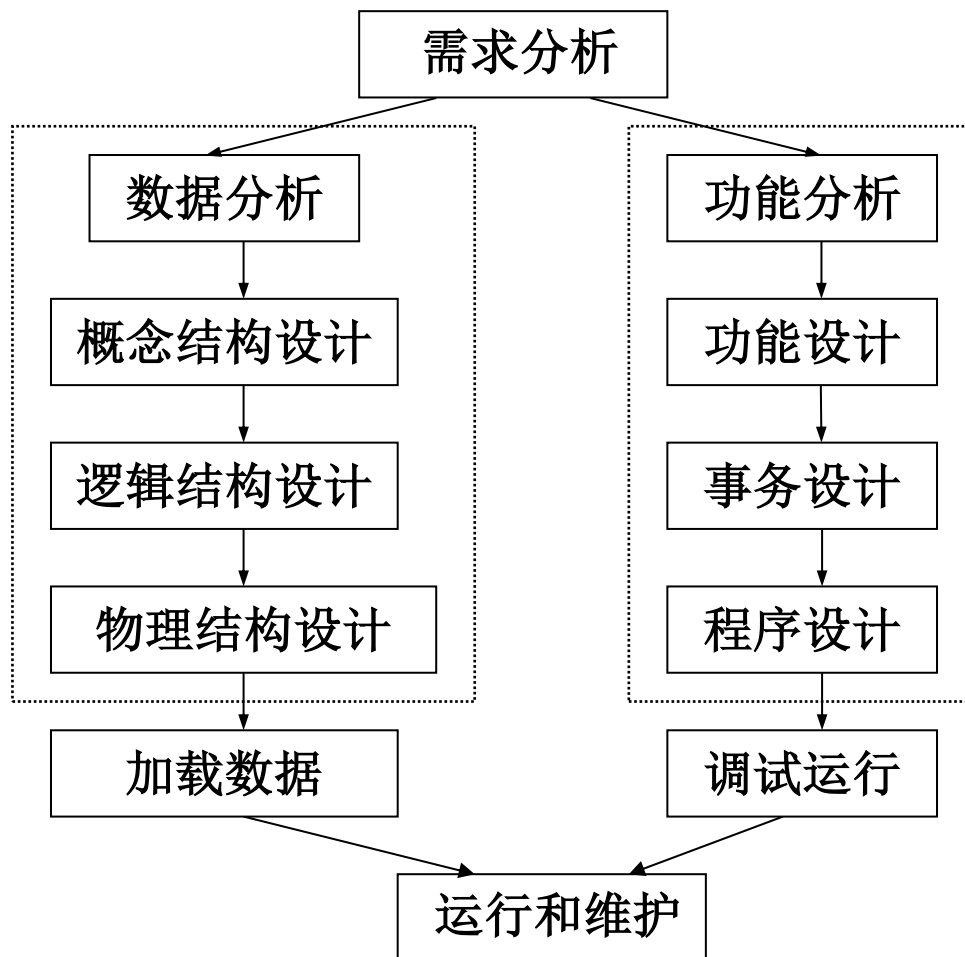
□设计一个完善的数据库应用系统往往是上述六个阶段的不断反复

数据库设计全过程



设计阶段	设计描述	
	数据	处理
需求分析	数据字典、全系统中数据项、数据流、数据存储的描述	数据流图和判定表（判定树）、数据字典中处理过程的描述
概念结构设计	概念模型（E-R图） 数据字典 	系统要求、方案和概图； 反映新系统信息流的数据流图 
逻辑结构设计	某种数据模型 	系统结构图 （模块结构） 
物理设计	存储安排 方法选择 存取路径建立 	模块设计 IPO表 
实施阶段	编写模式 装入数据 数据库试运行 	程序编码、编译联结、测试 
运行维护	性能监测、转储/恢复 数据库重组和重构	系统转换、运行、维护（修正性、适应性、改善性维护）

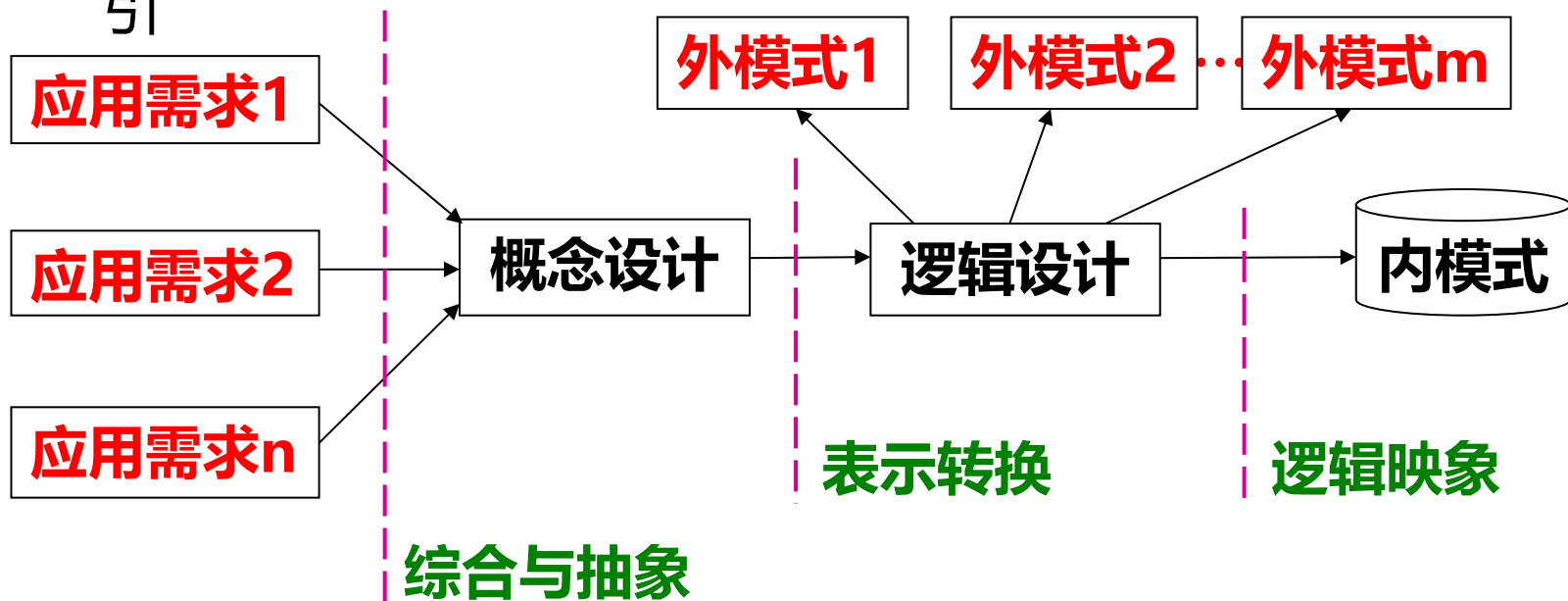
数据库设计全过程



数据库各级模式的形成过程

□数据库结构设计包括

- 需求分析阶段：综合各个用户的应用需求
- 概念结构设计：形成独立于各个DBMS概念模式，如E-R图
- 逻辑结构设计：形成数据库逻辑模式与外模式，用（基本）数据模型描述，例基本表、视图等
- 物理结构设计：形成数据库内模式，如DB文件或目录、索引



一. 需求分析

□需求分析就是分析用户的需要与要求

- 需求分析是设计数据库的起点
- 需求分析的结果是否准确反映了用户的实际要求，将直接影响到后面各个阶段的设计，并影响到设计结果是否合理和实用

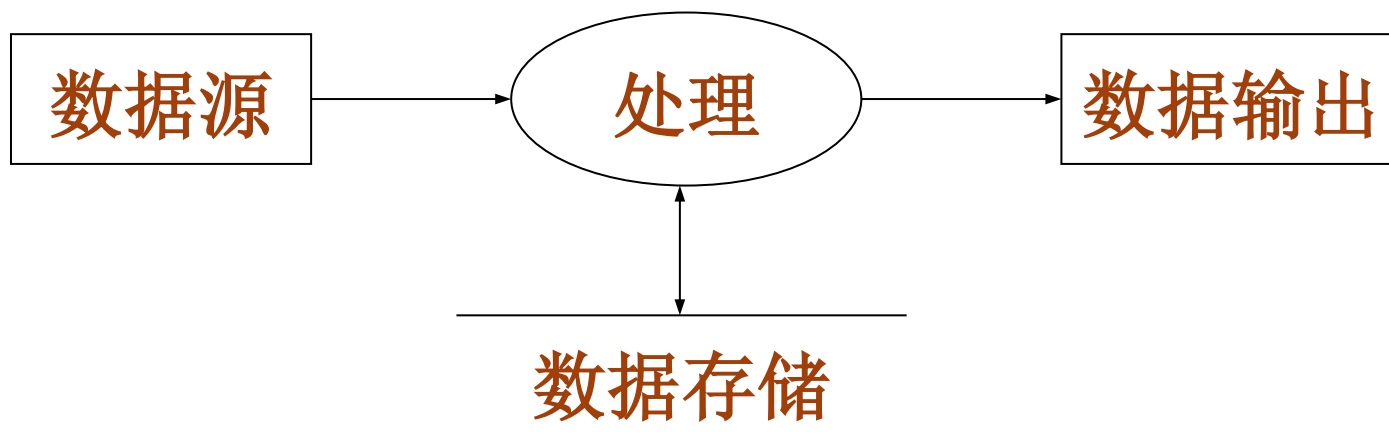
□分析和表达用户的需求的常用方法

- 自顶向下的结构化分析方法 (Structured Analysis, 简称SA方法)
- SA方法从最上层的系统组织机构入手，采用逐层分解的方式分析系统，并用数据流图和数据字典描述系统

进一步分析和表达用户需求



- (1) 在需求分析中，通过自顶向下、逐步分解的方法分析系统。任何一个系统都可以抽象为数据流图的形式



进一步分析和表达用户需求



□(2) 分解处理功能和数据

- 分解处理功能

- ◆ 将处理功能的具体内容分解为若干子功能，再将每个子功能继续分解，直到把系统的工作过程表达清楚

- 分解数据

- ◆ 在处理功能逐步分解的同时，其所用的数据也逐级分解，形成若干层次的数据流图。数据流图表达了数据和处理过程的关系

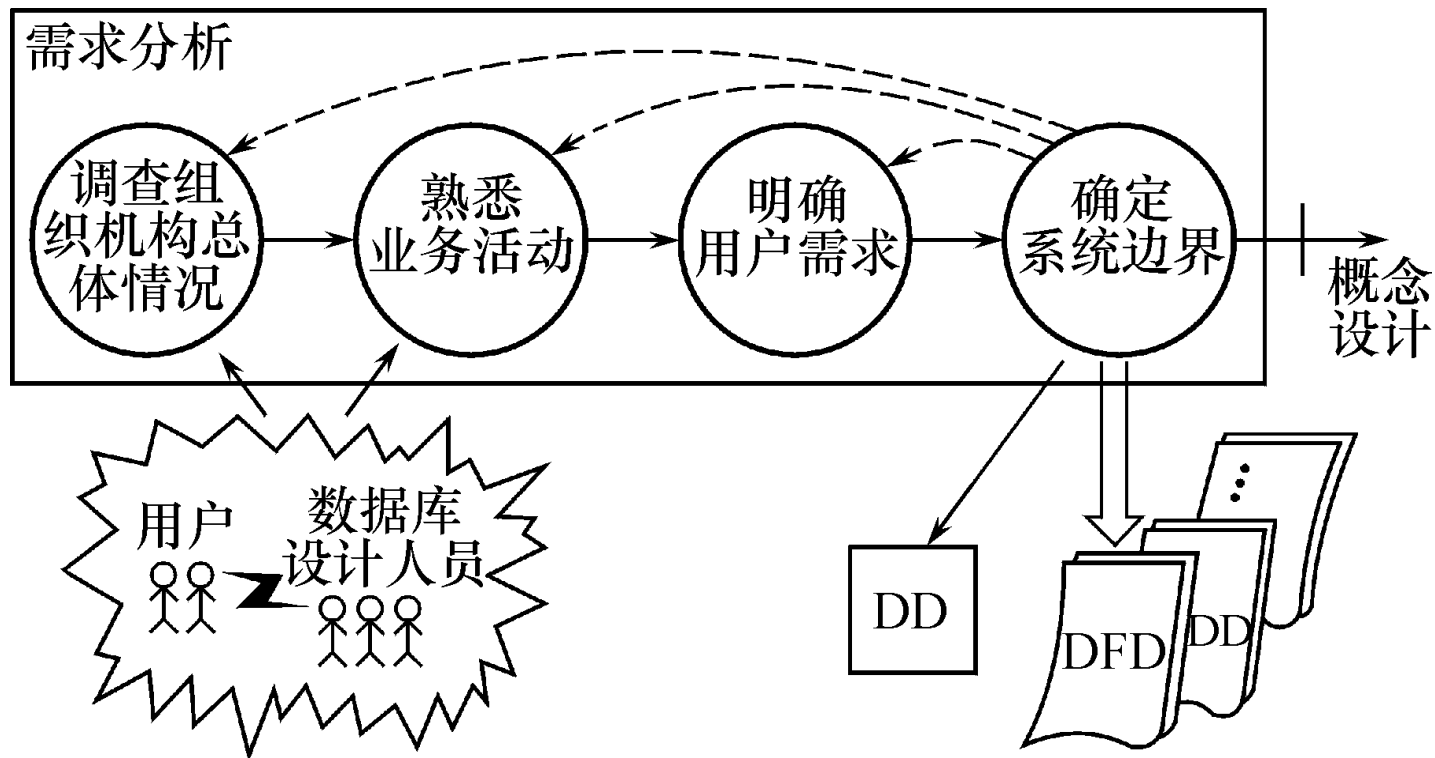
- 表达方法

- ◆ 处理过程：用判定表或判定树来描述
- ◆ 数据：用数据字典来描述

□(3) 将分析结果再次提交给用户，征得用户认可

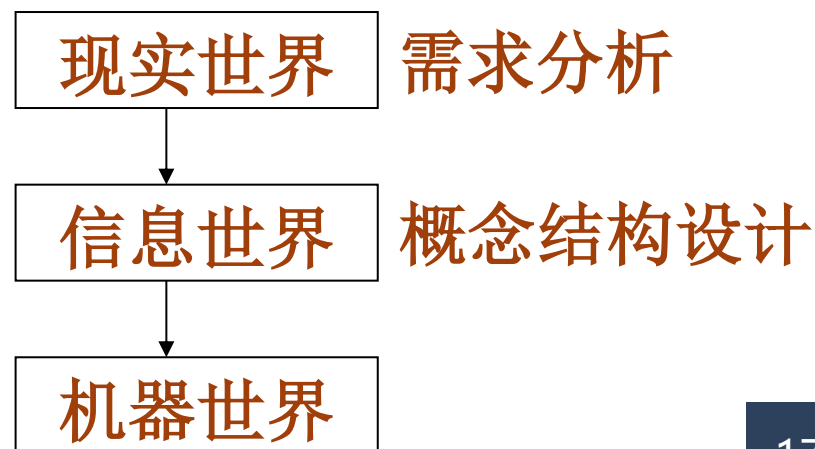
一. 需求分析

□ 下图描述了需求分析的过程



二. 概念设计

- 将需求分析得到的用户需求抽象为信息结构即概念模型的过程就是概念结构设计
 - 独立于数据模型（层次、网状、关系），更与采用的DBMS无关
- 概念结构是对现实世界的一种抽象
 - 实体
 - 实体的属性
 - 确定实体之间的联系类型
- 描述概念模型的工具
 - E-R模型

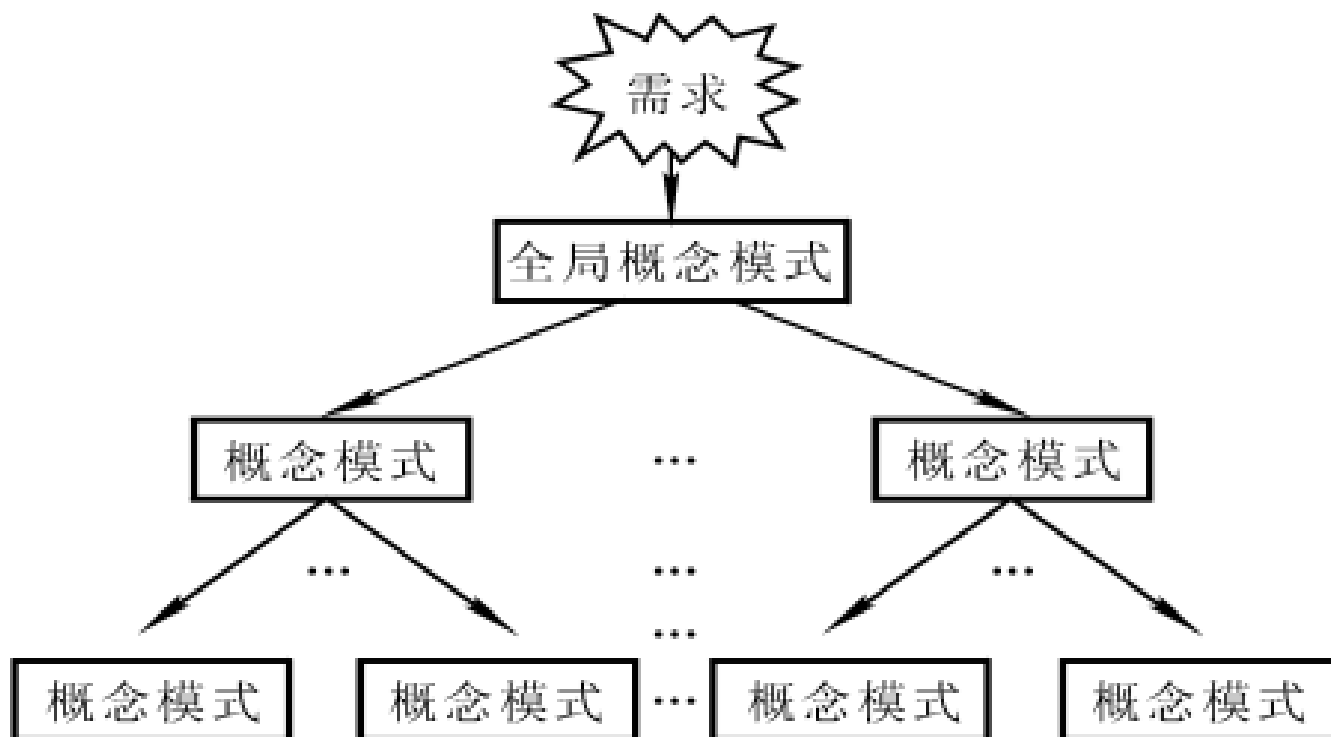


二. 概念设计

□设计概念结构的四类方法

- 自顶向下

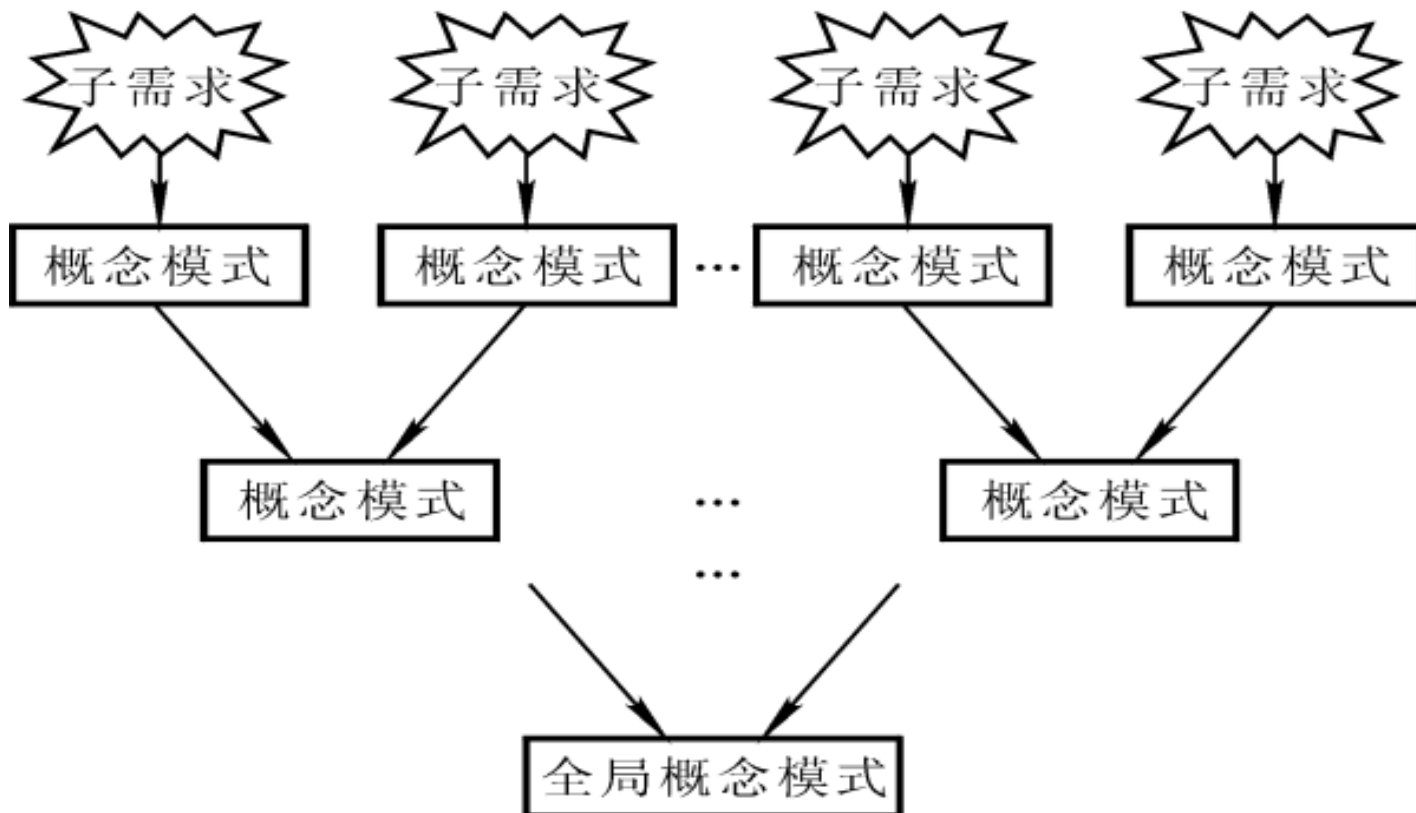
- ◆ 首先定义全局概念结构的框架，然后逐步细化



设计概念结构的四类方法

- 自底向上

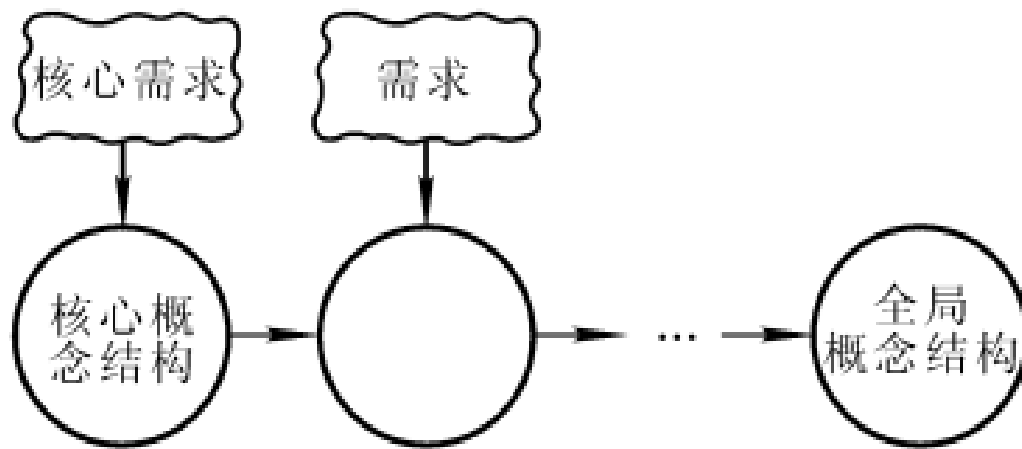
- ◆ 首先定义各局部应用的概念结构，然后将它们集成起来，得到全局概念结构



设计概念结构的四类方法

- 逐步扩张

- ◆ 首先定义最重要的核心概念结构，然后向外扩充，以滚雪球的方式逐步生成其他概念结构，直至总体概念结构



- 混合策略

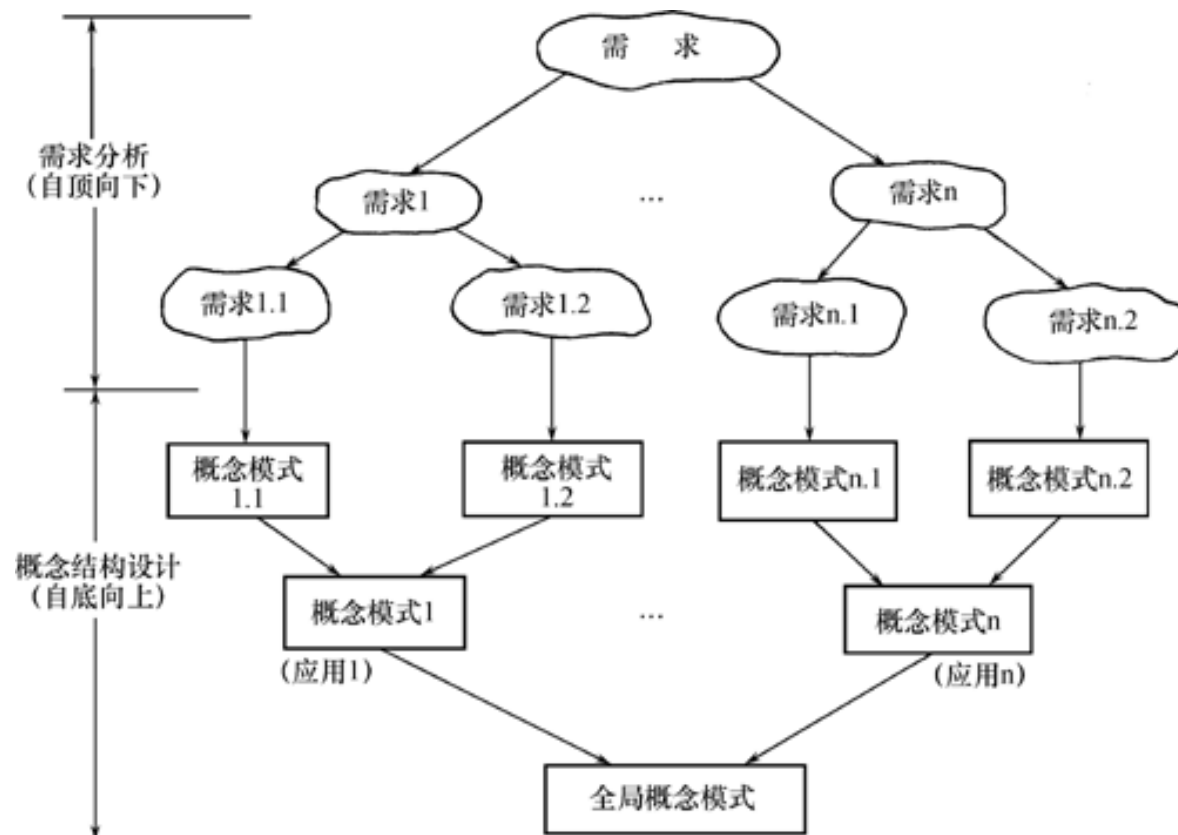
- ◆ 将自顶向下和自底向上相结合，用自顶向下策略设计一个全局概念结构的框架，以它为骨架集成由自底向上策略中设计的各局部概念结构

二. 概念设计



常用策略

- 自顶向下进行需求分析
- 自底向上设计概念结构



① 设计局部E-R模型

- E-R模型的设计内容包括确定局部E-R模型的范围、定义实体、联系以及它们的属性

② 设计全局E-R模型

- 将所有局部E-R图集成为一个全局E-R图，即全局E-R模型

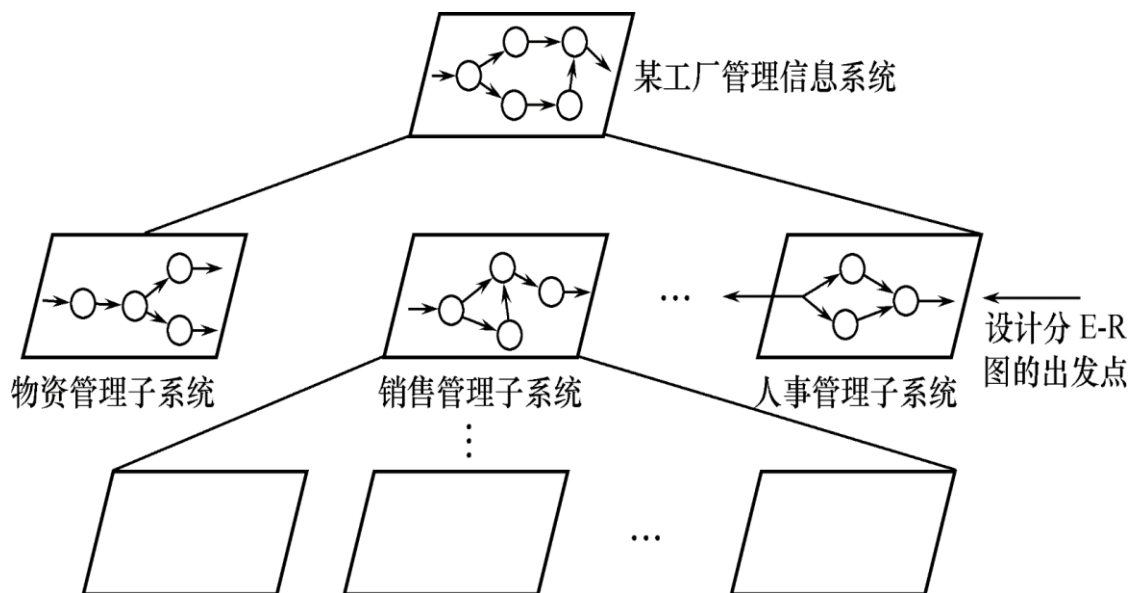
③ 验证整体概念结构

概念设计过程-①设计分E-R图



□选择局部应用

- 需求分析阶段已用多层数据流图和数据字典描述了整个系统
- 根据系统的具体情况，在多层的数据流图中选择一个适当层次的数据流图，作为设计分E-R图的出发点
- 通常以**中层数据流图**作为设计分E-R图的依据



概念设计过程-①设计分E-R图



□逐一设计分E-R图

- 将各局部应用涉及的数据分别从数据字典中抽取出来
- 参照数据流图，标定各局部应用中的实体、实体的属性、标识实体的码
- 确定实体之间的联系及其类型

概念设计过程-①设计分E-R图

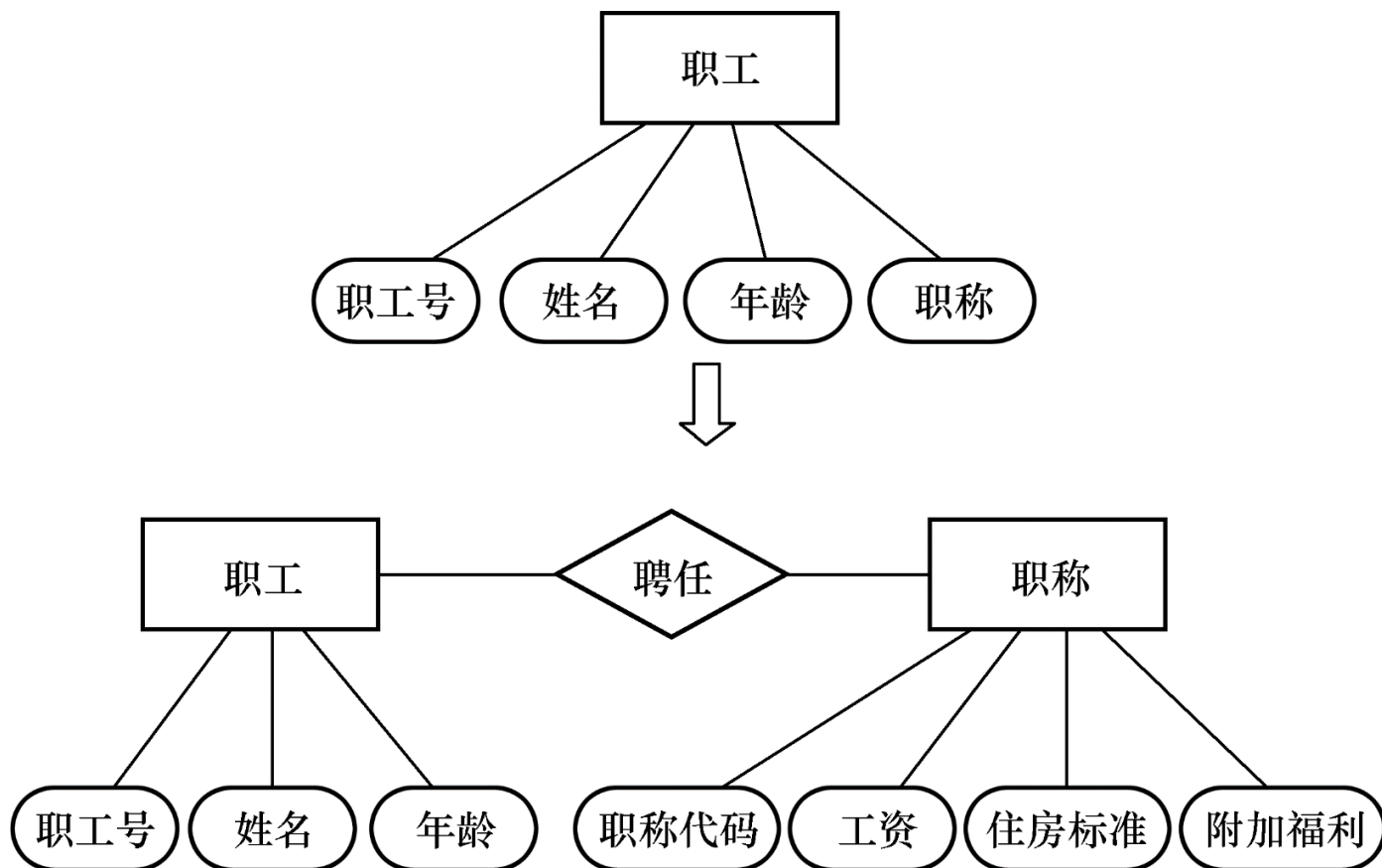


□如何抽象实体和属性？

- 实体：现实世界中一组具有某些共同特性和行为的对象就可以抽象为一个实体
- 属性：对象类型的组成成分可以抽象为实体的属性
- 实体与属性是相对而言的。同一事物，在一种应用环境中作为“属性”，在另一种应用环境中就必须作为“实体”
 - ◆ 例：学校中的系在某种应用环境中，它只是作为“学生”实体的一个属性，表明一个学生属于哪个系
 - ◆ 在另一种环境中，需要考虑一个系的系主任、教师人数、办公地点等，就需要作为实体

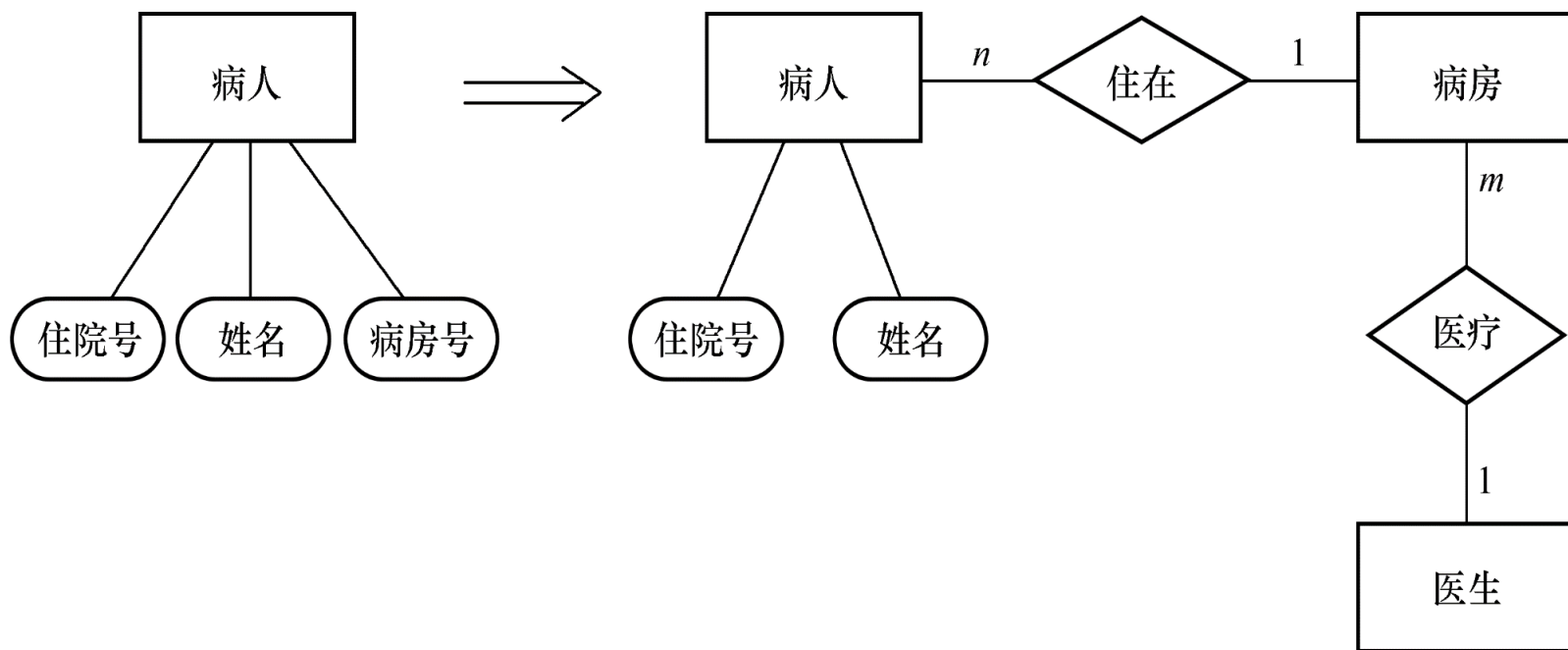
如何抽象实体和属性示例

- 职称作为一个属性
- 职称作为一个实体



如何抽象实体和属性示例

- 在医院中，一个病人只能住在一个病房，病房号可以作为病人实体的一个属性
- 但如果病房还要与医生实体发生联系，即一个医生负责几个病房的病人的医疗工作，则病房应作为一个实体



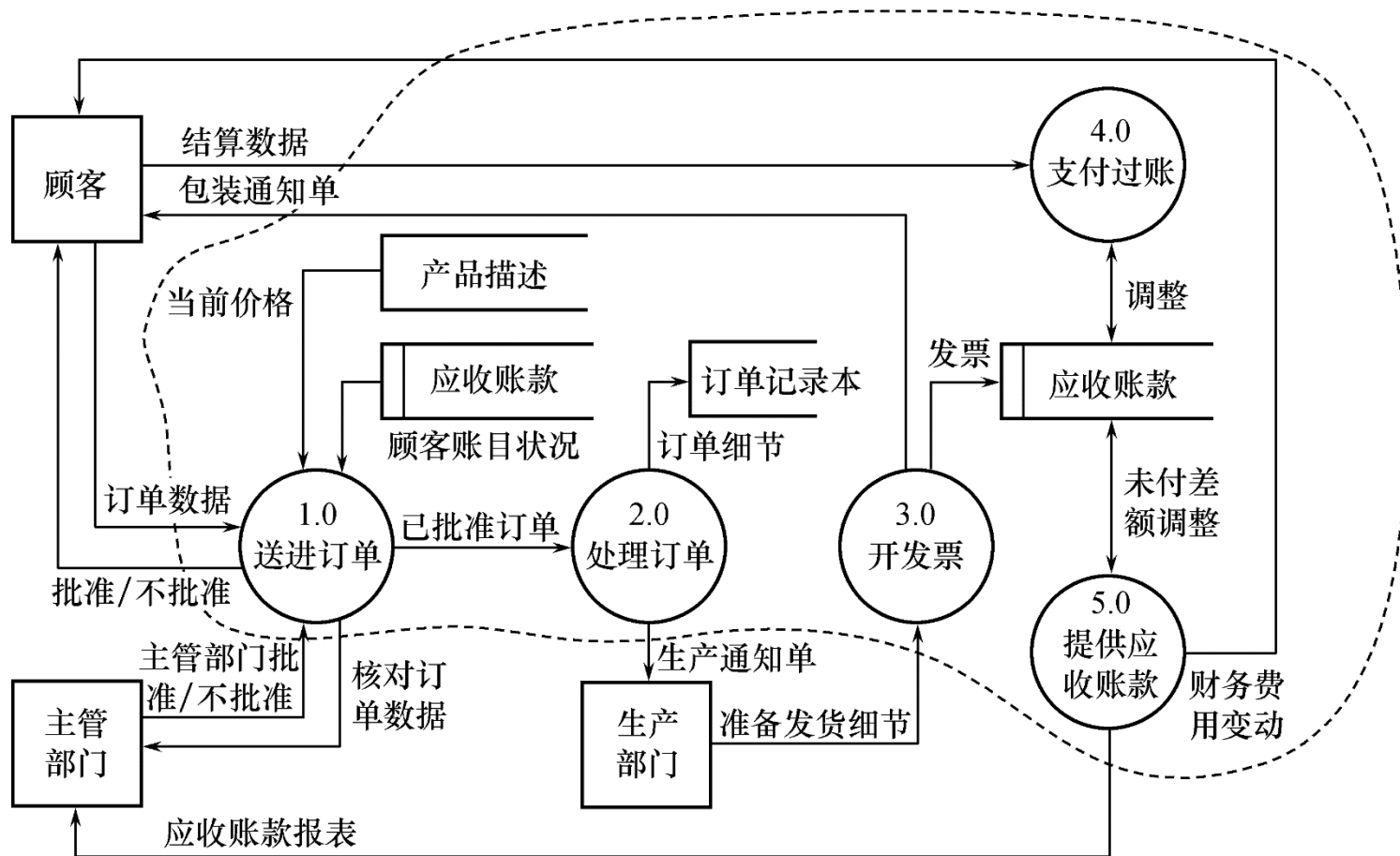
分E-R图设计示例

□[实例] 某工厂管理信息系统，其中销售管理子系统分E-R图的设计

- 销售管理子系统的主要功能：
 - ◆ 处理顾客和销售员送来的订单
 - ◆ 工厂是根据订货安排生产的
 - ◆ 交出货物同时开出发票
 - ◆ 收到顾客付款后，根据发票存根和信贷情况进行应收款处理

分E-R图设计示例

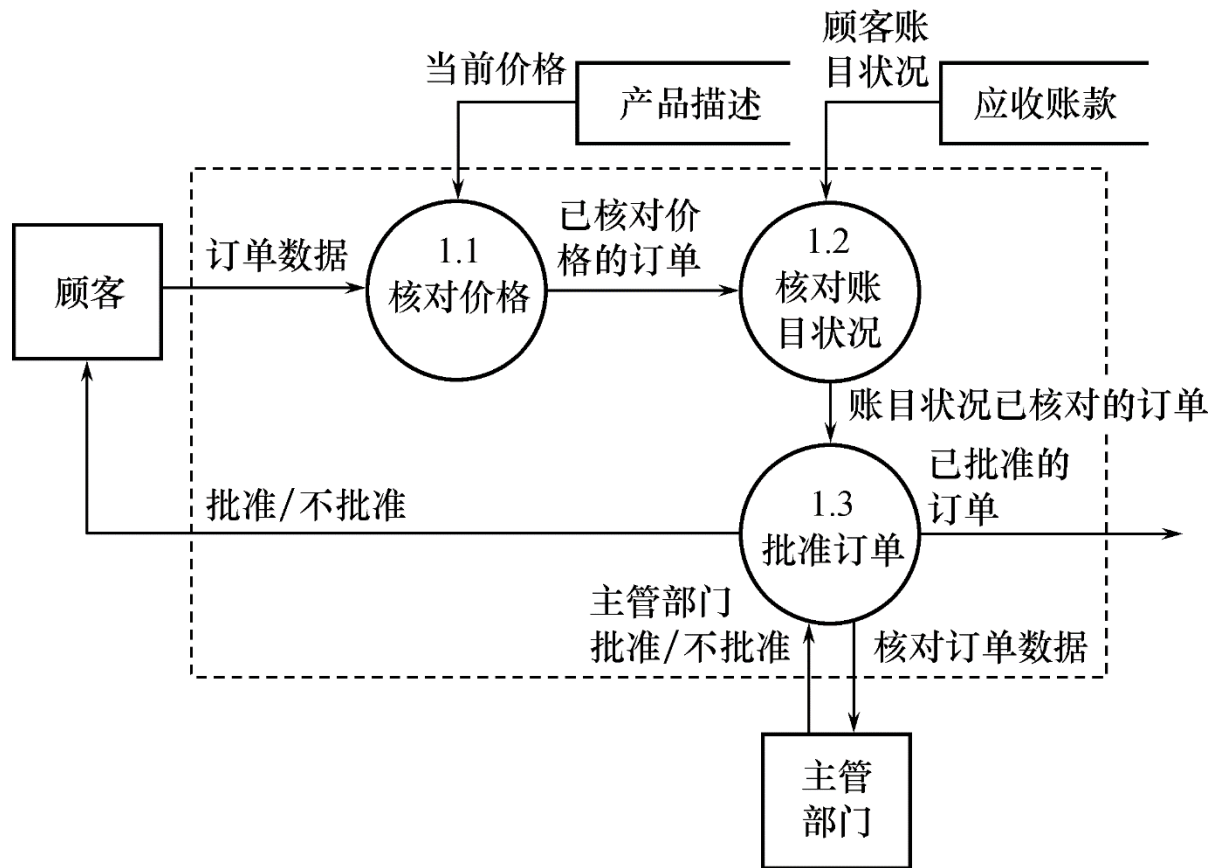
□销售管理子系统第一层数据流图，虚线部分划出了系统边界



分E-R图设计示例



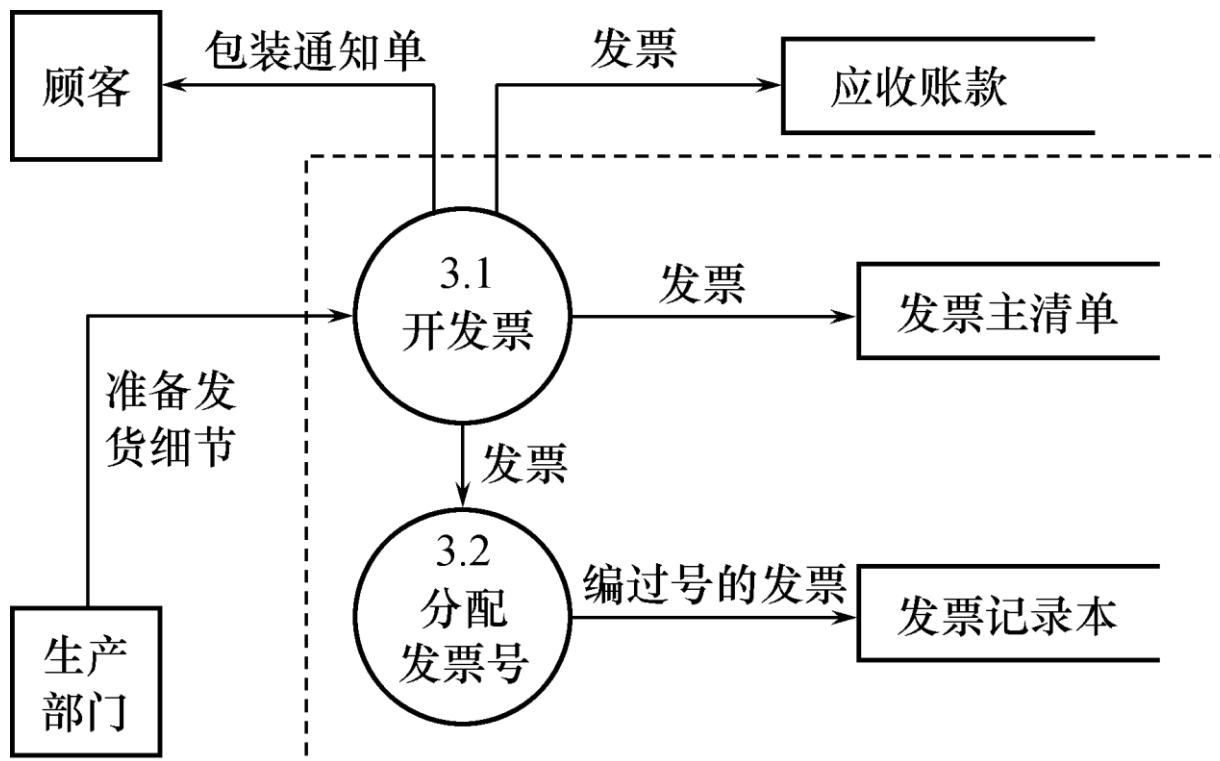
□ 子系统数据流图（第二层数据流图）



接收订单

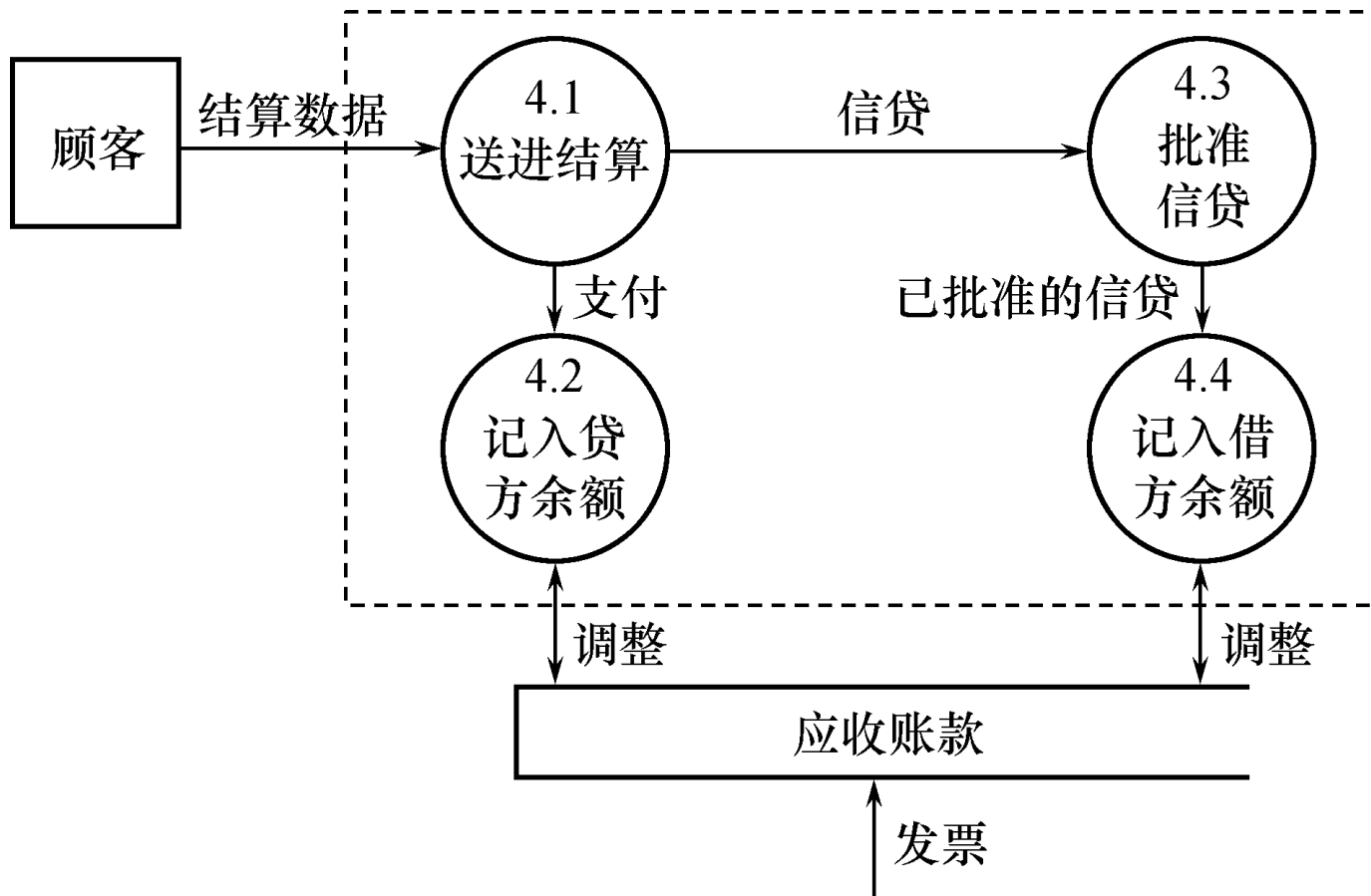


分E-R图设计示例



开发票

分E-R图设计示例

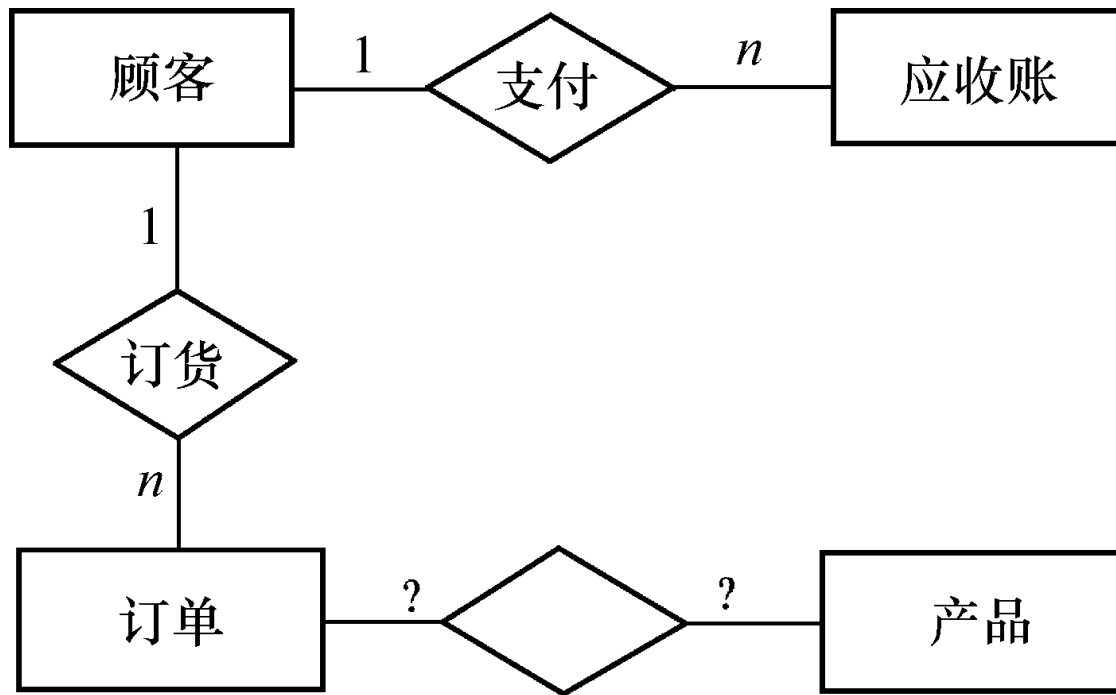


支付过账

分E-R图设计示例



□设计分E-R图框架

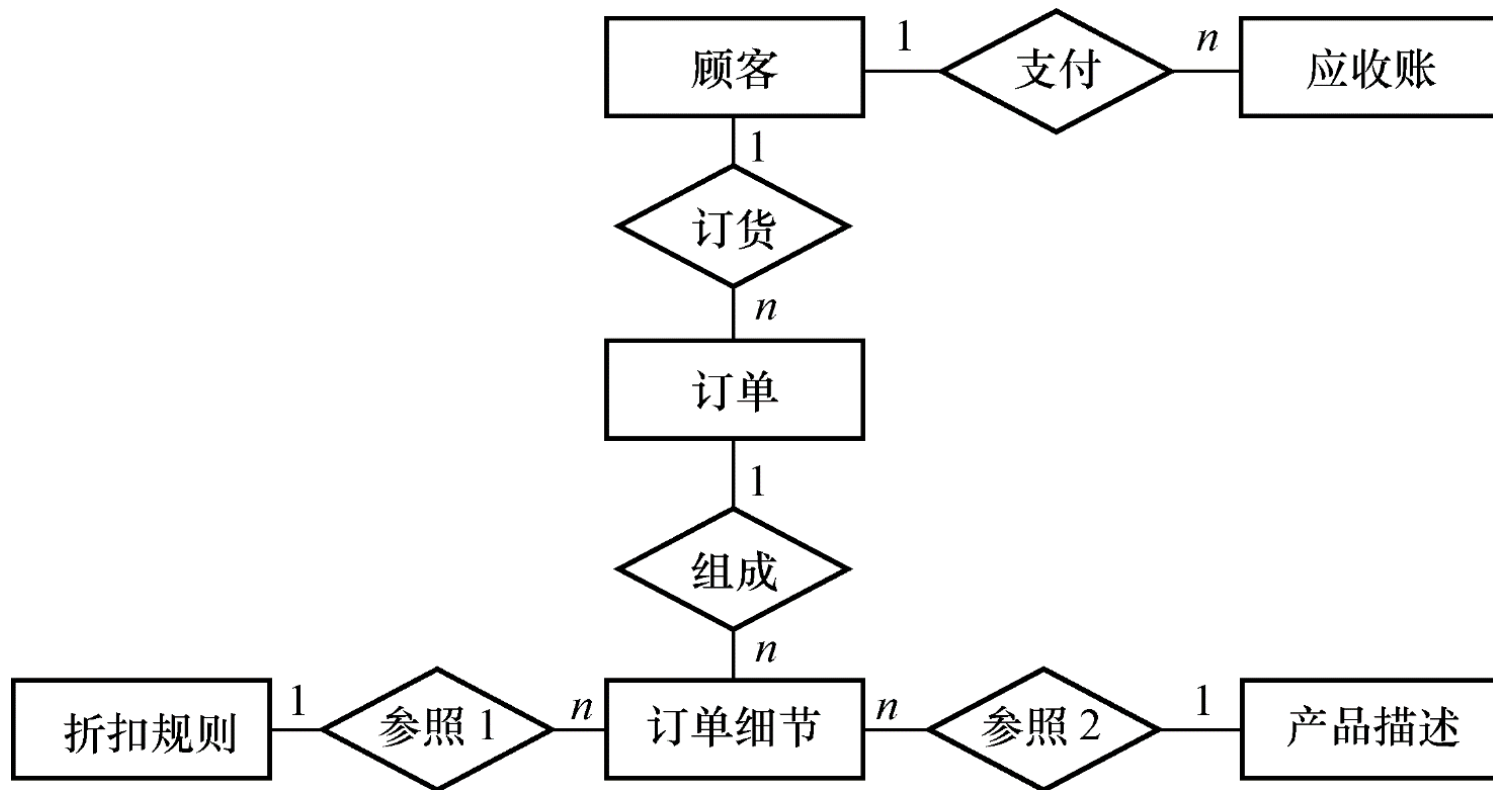


□进行如下调整：

- (1) 每张订单由订单号、若干头信息和订单细节组成。订单细节又有订货的零件号、数量等来描述
- (2) 订单细节就不能作订单的属性处理而应该上升为实体。一张订单可以订若干产品，所以订单与订单细节两个实体之间是1:n的联系
- (3) 原订单和产品的联系实际上是订单细节和产品的联系
- (4) 工厂对大宗订货给予优惠

分E-R图设计示例

□得到分E-R图如下图所示



分E-R图设计示例

□对每个实体定义的属性如下：

- 顾客：{顾客号，顾客名，地址，电话，信贷状况，账目余额}
- 订单：{订单号，顾客号，订货项数，订货日期，交货日期，工种号，生产地点}
- 订单细则：{订单号，细则号，零件号，订货数，金额}
- 应收账款：{顾客号，订单号，发票号，应收金额，支付日期，支付金额，当前余额，货款限额}
- 产品描述：{产品号，产品名，单价，重量}
- 折扣规则：{产品号，订货量，折扣}

概念设计过程-②集成局部E-R图



- 各个局部视图即分E-R图建立好后，还需要对它们进行合并，集成为一个整体的数据概念结构即总E-R图
 - 逐步累积式
 - ◆ 首先集成两个局部视图（通常是比较关键的两个局部视图）
 - ◆ 以后每次将一个的新的局部视图集成进来



□集成局部E-R图的步骤

- 1) 合并分E-R图，生成初步E-R图
- 2) 通过修改与重构，消除不必要的冗余，得到基本E-R图

概念设计过程-②集成局部E-R图



□ 1) 合并分E-R图，生成初步E-R图

- 解决各分E-R图之间的冲突，将各分E-R图合成初步E-R图，解决冲突是合并E-R图的主要工作和关键所在
- 冲突主要有三类
 - ◆ **属性冲突**：属性域冲突、属性取值单位冲突
 - ◆ **命名冲突**：同名异义和异名同义
 - ◆ **结构冲突**：同一对象在不同应用中具有不同的抽象、同一实体在不同的局部E-R图所包含的属性个数和属性的排列次序不完全相同



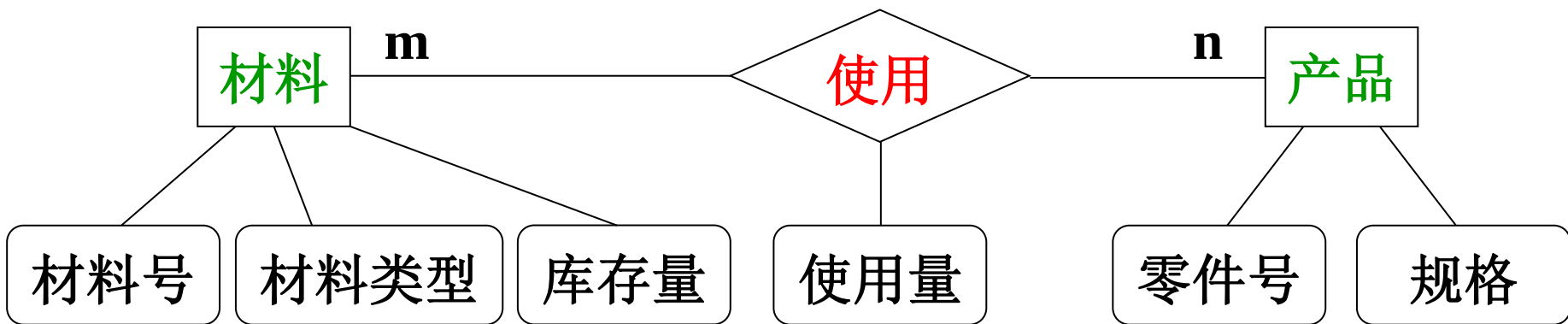
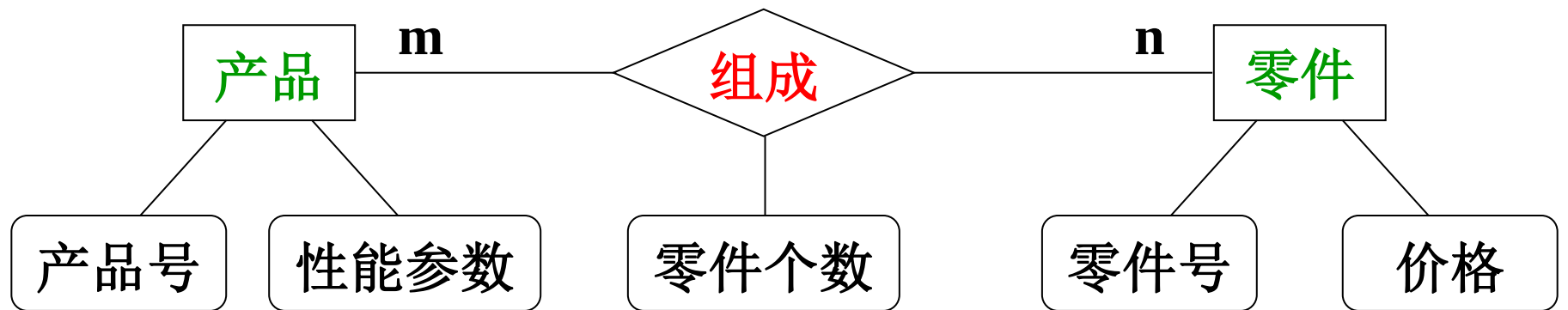
- 2) 通过修改与重构，消除不必要的冗余，得到基本E-R图
 - 初步合并得到的E-R图，可能存在冗余的数据和冗余的实体间联系
 - 冗余数据和冗余联系容易破坏数据库的完整性，给数据库维护增加困难
 - 冗余的消除方法
 - ◆ 采用分析方法，通过分析数据项之间逻辑关系来消除冗余数据
 - ◆ 规范化理论中函数依赖的概念提供了消除冗余联系的形式化工具

概念设计过程-③验证整体概念结构

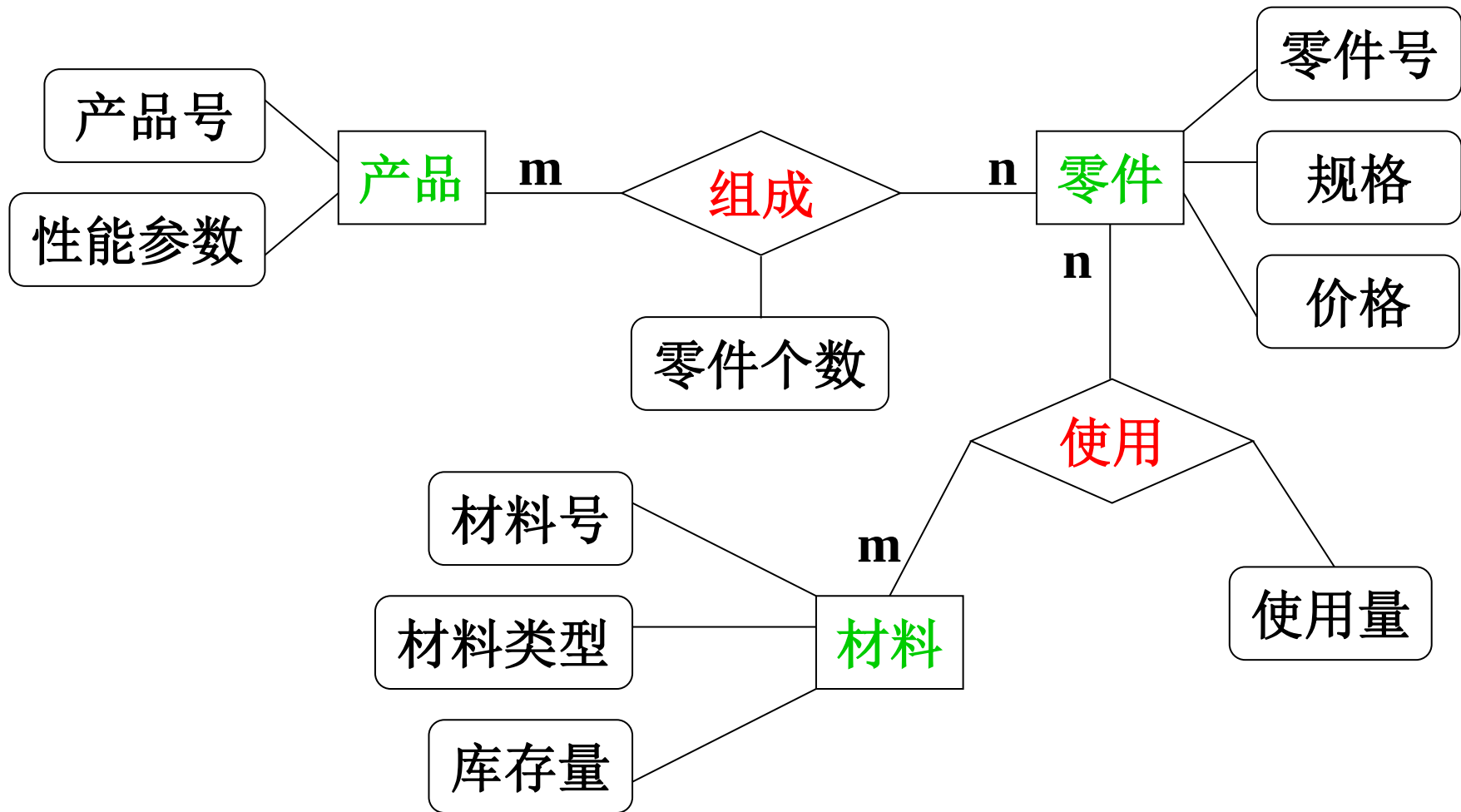


- 视图集成后形成一个整体的数据库概念结构，必须进行进一步验证，确保它能够满足下列条件：
 - 整体概念结构内部必须具有一致性，不存在互相矛盾的表达
 - 整体概念结构能准确地反映原来的每个视图结构，包括属性、实体及实体间的联系
 - 整体概念结构能满足需求分析阶段所确定的所有要求
- 整体概念结构最终还应该提交给用户，征求用户和有关人员的意见，进行评审、修改和优化，然后才能作为进一步设计数据库的依据

局部E-R图



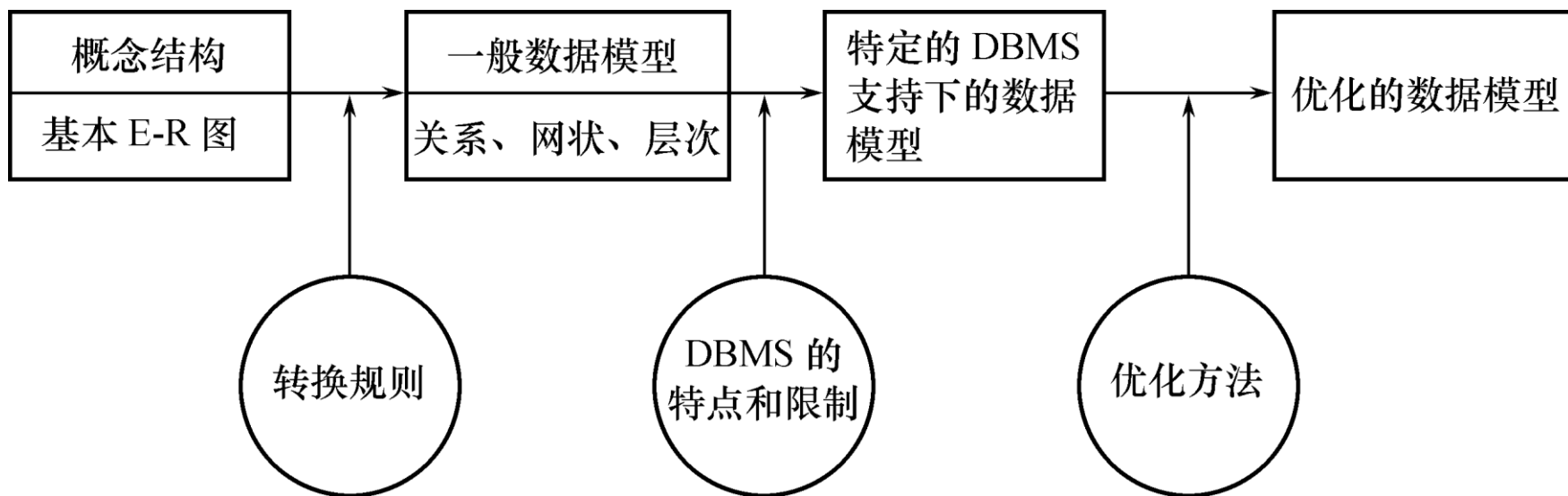
合并示例



三. 逻辑结构设计

- 把概念结构设计阶段设计好的基本E-R模型转换为具体的数据库管理系统支持的数据模型
 - 也就是导出特定的DBMS可以处理的数据库逻辑结构（数据库的模式和外模式），这些模式在功能、性能、完整性和一致性约束方面满足应用要求
- 逻辑结构设计的步骤
 - 将概念结构转化为一般的关系、网状、层次模型
 - 将转换来的关系、网状、层次模型向特定DBMS支持下的数据模型转换
 - 对数据模型进行优化

三. 逻辑结构设计



E-R模型向关系模型的转换

- 一个实体转换为一个关系模式。实体的属性就是关系的属性，实体的标识符就是关系的码
- 对于实体间的联系有以下不同的情况
 - 一个1:1联系可以转换为一个独立的关系模式，也可以与任意一端所对应的关系模式合并
 - 一个1:n联系可以转换为一个独立的关系模式，也可以与n端所对应的关系模式合并
 - 一个m:n联系转换为一个独立的关系模式
 - 三个或三个以上实体间的一个多元联系可以转换为一个关系模式
 - 具有相同码的关系模式可以合并

1:1转换示例

□ 一个1:1联系转换为一个独立的关系模式

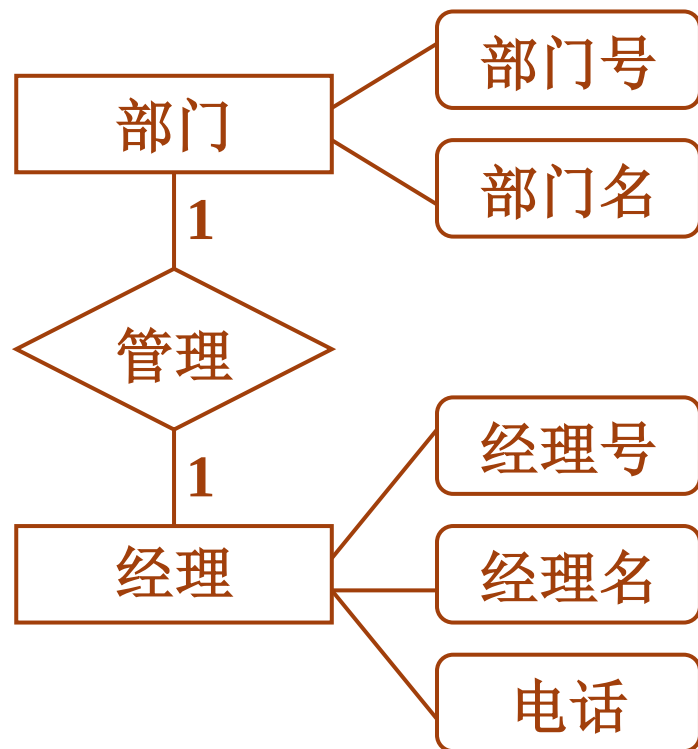
- 部门表 (部门号, 部门名)
- 经理表 (经理号, 经理名, 电话)
- 部门-经理 (经理号, 部门号)

□ 也可以与任意一端所对应的关系模式合并, 需要在该关系模式的属性中加入另一个实体的码和联系本身的属性

- 部门表 (部门号, 部门名, 经理号)
- 经理表 (经理号, 经理名, 电话)

或者:

- 部门表 (部门号, 部门名)
- 经理表 (经理号, 部门号, 经理名, 电话)



1:n转换示例

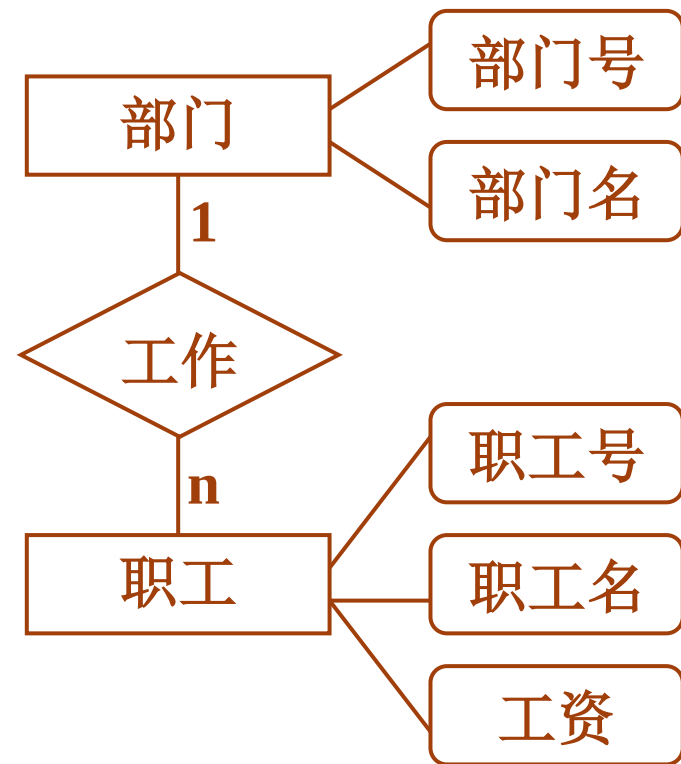
□ 一个1:n联系可以转换为一个独立的关系模式

- 与该联系相连的各实体的码以及联系本身的属性均转换为此关系模式的属性，且关系模式的码为n端实体的码

- 部门表 (部门号, 部门名)
- 职工表 (职工号, 职工名, 工资)
- 部门-职工 (部门号, 职工号)

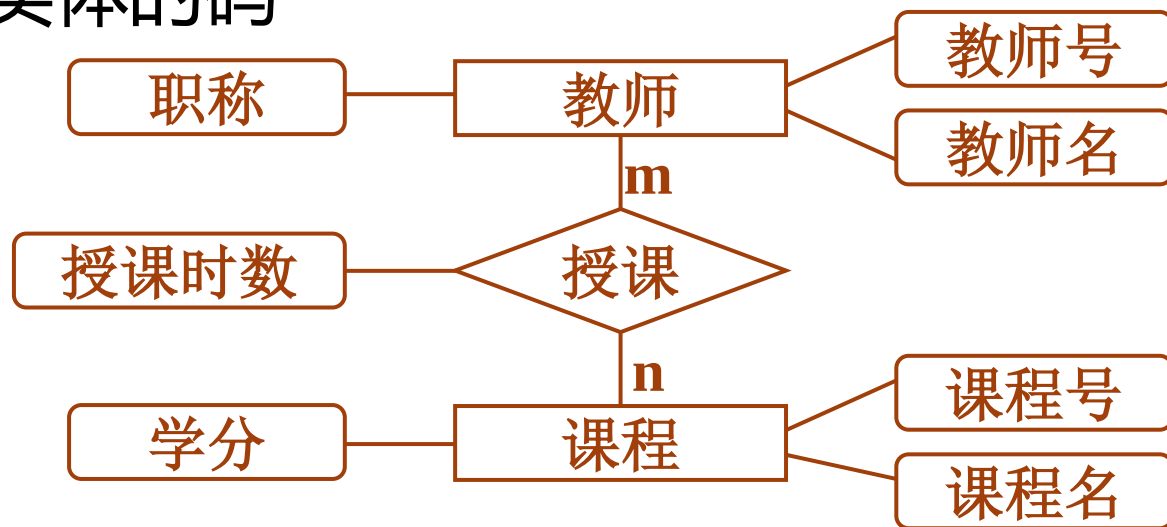
□ 也可以与n端所对应的关系模式合并，需要将该关系模式中加入1端实体的码以及联系本身的属性

- 部门表 (部门号, 部门名)
- 职工表 (职工号, 部门号, 职工名, 工资)



m:n转换示例

- 一个 **m:n联系** 转换为一个独立的关系模式。与该联系相连的各实体的码以及联系本身的属性均转换为此关系模式的属性，而此关系模式的主码包含各实体的码



- 教师表 (教师号, 教师名, 职称)
- 课程表 (课程号, 课程名, 学分)
- 授课表 (教师号, 课程号, 授课时数)

m:n转换示例

- 三个或三个以上实体间的一个多元联系转换为一个关系模式
- **[例]** “讲授”联系是一个三元联系，可以将它转换为如下关系模式，其中课程号、职工号和书号为关系的组合码：

讲授 (课程号, 职工号, 书号)

E-R图向关系模型的转换

□具有相同键的关系模式可合并

- 目的：减少系统中的关系个数
- 合并方法：将其中一个关系模式的全部属性加入到另一个关系模式中，然后去掉其中的同义属性（可能同名也可能不同名），并适当调整属性的次序

E-R图向关系模型的转换

ER – 联系类型约束		关系模型 – 表
1:1 键约束	参与实体	关系表
	联系 →	独立的关系表
	联系 →	一端实体的主键和联系属性移到另一端实体的关系表中
1:n 键约束	参与实体	关系表
	联系 →	独立的关系表
	联系 →	“1”端实体的主键和联系属性移动到“n”端实体的关系表中
m:n 键约束	参与实体	关系表
	联系 →	独立的关系表，由参与实体的主键、联系的属性共同构成

数据模型的优化

- 数据库逻辑设计的结果不是唯一的，还应该适当地修改和调整，这是数据模型的优化
- 关系数据模型的优化通常以规范化理论为指导，并考虑系统的性能
 - 确定各属性间的数据依赖
 - 消除冗余的联系
 - 确定最合适的范式
 - 确定是否要对某些模式进行分解或合并

数据模型的优化

1. 确定数据依赖

- 按需求分析阶段所得到的语义，分别写出每个关系模式内部各属性之间的数据依赖以及不同关系模式属性之间的数据依赖

2. 对于各个关系模式之间的数据依赖进行极小化处理，消除冗余的联系

3. 按照数据依赖的理论对关系模式逐一进行分析，考查是否存在部分函数依赖、传递函数依赖、多值依赖等，确定各关系模式分别属于第几范式

4. 按照需求分析阶段得到的各种应用对数据处理的要求，分析对于这样的应用环境这些模式是否合适，确定是否要对它们进行合并或分解

数据模型的优化

□并不是规范化程度越高的关系就越优

- 当一个应用的查询中经常涉及到两个或多个关系模式的属性时，系统必须经常地进行连接运算，而连接运算的代价是相当高的，可以说关系模型低效的主要原因就是做连接运算引起的，因此在这种情况下，第二范式甚至第一范式也许是最好的
- 非BCNF的关系模式虽然从理论上分析会存在不同程度的更新异常，但如果在实际应用中对此关系模式只是查询，并不执行更新操作，就不会产生实际影响
- 对于一个具体应用来说，到底规范化进行到什么程度，需要权衡响应时间和潜在问题两者的利弊才能决定。一般说来，第三范式就足够了

数据模型的优化

□例：在关系模式

学生成绩单 (学号, 英语, 数学, 语文, 平均成绩)

中存在下列函数依赖：

学号→英语

学号→数学

学号→语文

学号→平均成绩

(英语, 数学, 语文)→平均成绩

数据模型的优化

显然有：

学号 $\rightarrow$ (英语, 数学, 语文)

因此该关系模式中存在传递函数依赖，是2NF关系

- 虽然平均成绩可以由其他属性推算出来，但如果应用中需要经常查询学生的平均成绩，为提高效率，仍然可保留该冗余数据，对关系模式不再做进一步分解

数据模型的优化

5. 按照需求分析阶段得到的各种应用对数据处理的要求，对关系模式进行必要的分解，以提高数据操作的效率和存储空间的利用率

- 常用分解方法
 - ◆ 水平分解
 - ◆ 垂直分解

数据模型的优化

□ 水平分解

- 什么是水平分解

- ◆ 把关系的元组分为若干子集合，定义每个子集合为一个子关系，以提高系统的效率

- 水平分解的适用范围

- ◆ 满足“8/2原则”的应用

- ◆ 并发事务经常存取不相交的数据

□ 垂直分解

- 什么是垂直分解
 - ◆ 把关系模式 R 的属性分解为若干子集合，形成若干子关系模式
- 垂直分解的适用范围
 - ◆ 取决于分解后 R 上的所有事务的总效率是否得到了提高

设计用户子模式

- 定义数据库模式主要是从系统的时间效率、空间效率、易维护等角度出发
- 外模式概念对应关系数据库的视图概念，设计外模式是为了更好地满足局部用户的需求
- 定义用户外模式时应该更注重考虑用户的习惯与方便。包括三个方面：
 1. 使用更符合用户习惯的别名
 2. 针对不同级别的用户定义不同的外模式，以满足系统对安全性的要求
 3. 简化用户对系统的使用

□任务

- 将概念结构转化为具体的数据模型

□逻辑结构设计的步骤

- 将概念结构转化为一般的关系、网状、层次模型
- 将转化来的关系、网状、层次模型向特定DBMS支持下的数据模型转换
- 对数据模型进行优化
- 设计用户子模式

四. 数据库的物理设计

□什么是数据库的物理设计

- 数据库在物理设备上的存储结构与存取方法称为数据库的物理结构，它依赖于给定的计算机系统
- 为一个给定的逻辑数据模型选取一个最适合应用环境的物理结构的过程，就是数据库的物理设计

□数据库物理设计的步骤

- 确定数据库的物理结构
- 对物理结构进行评价，评价的重点是时间和空间效率
- 如果评价结果满足原设计要求则可进入到物理实施阶段，否则，就需要重新设计或修改物理结构，有时甚至要返回逻辑设计阶段修改数据模型

数据库物理设计的内容和方法



- 选择物理数据库设计所需参数，是确定关系存取方法的依据
 - 数据库查询事务
 - ◆ 查询的关系
 - ◆ 查询条件所涉及的属性
 - ◆ 连接条件所涉及的属性
 - ◆ 查询的投影属性
 - 数据更新事务
 - ◆ 被更新的关系
 - ◆ 每个关系上的更新操作条件所涉及的属性
 - ◆ 修改操作要改变的属性值
 - 每个事务在各关系上运行的频率和性能要求

数据库物理设计的内容和方法



□ 关系数据库物理设计的内容

- 1. 为关系模式选择存取方法（建立存取路径）
- 2. 设计关系、索引等数据库文件的物理存储结构

□ 物理设计的必要工作

- 除了要实现概念模型向DBMS的转化的任务
- 同时加入概念模型中未考虑或未全部考虑的因素
 - ◆ PK (主键), FK (外键), View (视图), Index (索引), Trigger (触发器), Stored Procedure (存储过程) 等, 以实现系统的完整性

确定存取方法

- 一般用户可以通过建立索引的方法来加快数据的查询效率
- 建立索引的一般原则为：
 - 如果一个（或一组）属性经常在查询条件中出现，则考虑在这个（或这组）属性上建立索引（或组合索引）
 - 如果一个属性经常作为最大值和最小值等聚集函数的参数，则考虑在这个属性上建立索引
 - 如果一个（或一组）属性经常在连接操作的连接条件中出现，则考虑在这个（或这组）属性上建立索引
- 一个表可以建立多个索引，但只能建立一个聚簇索引

确定存储结构

□一般的存储方式有：

- 顺序存储
- 散列存储
- 聚簇存储

□一般情况下系统都会为数据选择一种最合适的存储方式

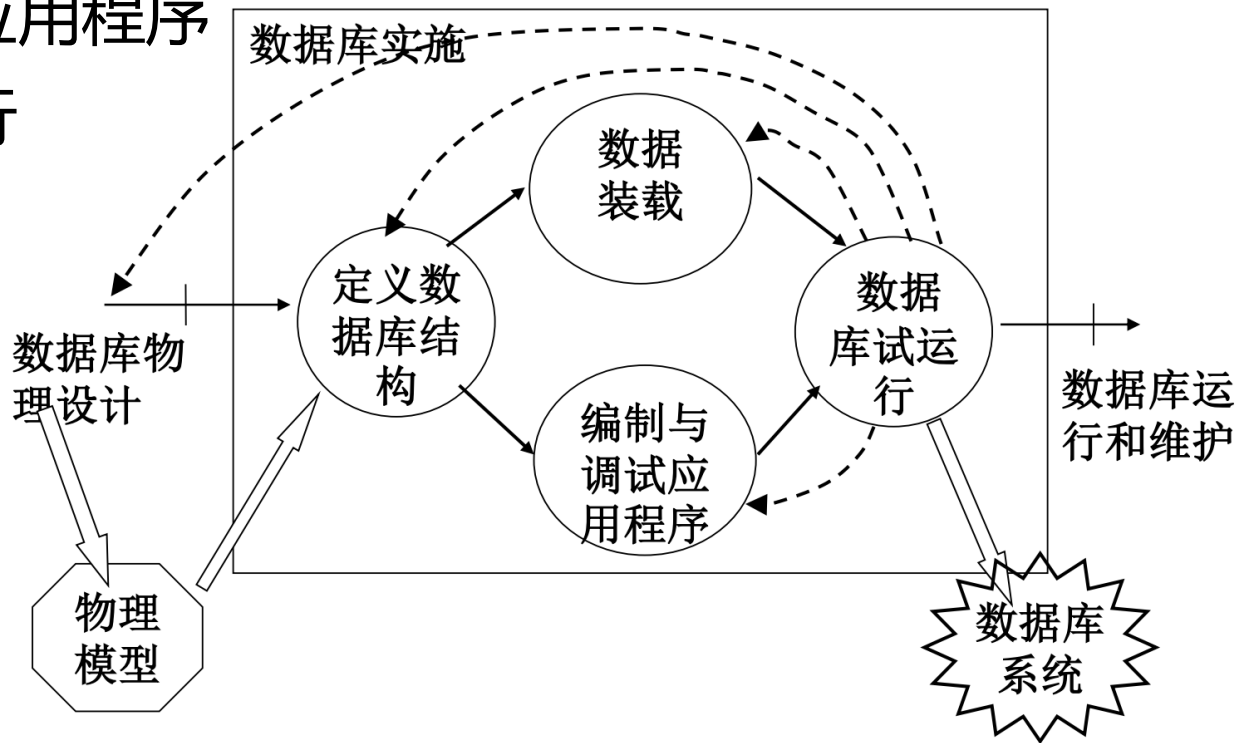
物理结构设计的评价

- 对数据库物理设计过程中产生的多种方案进行评价，从中选择一个较优的方案作为数据库的物理结构
- 评价方法
 - 定量估算各种方案
 - ◆ 存储空间
 - ◆ 存取时间
 - ◆ 维护代价
 - 对估算结果进行权衡、比较，选择出一个较优的合理的物理结构
 - 如果该结构不符合用户需求，则需要修改设计

五. 数据库的实施

□数据库实施的工作内容

- 用DDL定义数据库结构
- 组织数据入库
- 编制与调试应用程序
- 数据库试运行



六. 数据库的运行和维护

- 在数据库运行阶段，对数据库经常性的维护工作主要是由DBA来完成
 - 数据库的转储和恢复
 - 数据库的安全性、完整性控制
 - 数据库性能的监督、分析和改造
 - ◆ 性能调整或数据库重构
 - 数据库的重组织与重构造
 - ◆ 修改模式或外模式
 - ◆ 当系统发生重大变化，重构也无济于事，则数据库系统生命周期结束

六. 数据库的运行和维护

- 数据库的再组织是指按原设计要求重新安排存储位置、回收垃圾、减少指针链等，以提高系统性能
- 数据库运行一段时间后，由于记录不断增、删、改，会使数据库的物理存储情况变坏，降低了数据的存取效率，数据库性能下降，这时DBA就要对数据库进行重组织。DBMS一般都提供数据重组织用的实用程序
- 数据库的重组织与重构造
 - 数据库的重构造则是指部分修改数据库的模式和内模式，即修改原设计的逻辑和物理结构。数据库的再组织是不修改数据库的模式和内模式的



本章小结

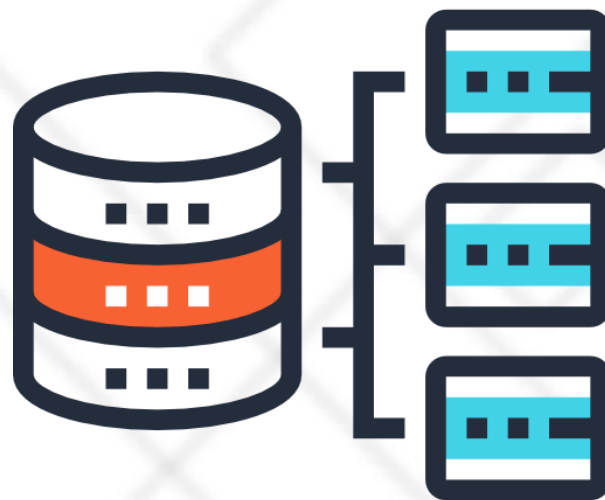


□ 数据库设计六个步骤

- (1) 需求分析
- (2) 概念结构设计
- (3) 逻辑结构设计
- (4) 数据库物理设计
- (5) 数据库实施
- (6) 数据库运行和维护



北京理工大学
BEIJING INSTITUTE OF TECHNOLOGY



思考题

《数据库系统原理》第11章

习题12

给出概念设计和逻辑设计的结果（采用关系模型）